When AES Throughput Depends on the CPU Execution Path: Evidence from x86-64 and Arm64

Marko Mihajlović[1]

Abstract: Reported encryption throughput is not an intrinsic property of AES: it depends on instruction support, cryptographic-library dispatch, and the interval selected for timing. This paper re-examines existing, validated CPU measurements from a Windows x86-64 laptop and an Oracle Cloud Arm64 instance. In 32 MiB AES-128-CTR measurements, native OpenSSL EVP reaches approximately 4.03 GB/s on Arm64, versus 0.21–0.22 GB/s for the inspected alternative paths. On x86-64, accounting for allocation and output conversion substantially reduces the apparent gain of native encryption. Historical CBC and authenticated GCM measurements provide an independent perspective on key length, message size, and verification costs. The results support boundary-aware CPU benchmarking but do not establish instruction-level causation or a platform-wide architecture ranking.

Keywords: AES; CPU instruction acceleration; OpenSSL EVP; PyCryptodome; Arm64; authenticated encryption; benchmarking.

1 Introduction

The Advanced Encryption Standard (AES) is used across data storage, software services, embedded devices and communication systems. Its standardized round operations are independent of processor choice, but its measured execution time is not. Modern CPUs may use dedicated cryptographic instructions; software libraries dispatch to different implementations; application frameworks may allocate output storage, initialize contexts and marshal data at different points. A quoted throughput figure can therefore reflect a cryptographic primitive, an application wrapper, or a combination of both.

This ambiguity becomes particularly pronounced when implementations cross a foreign-function boundary. A native library can process an already allocated buffer with a preinitialized context, whereas a Python-facing API can include argument handling and allocation. Comparing the resulting durations without stating their scopes risks turning a claim about a specific API into an unsupported claim about an algorithm or processor. Conversely, measuring only the native transform underestimates the elapsed time of a caller that must assemble a complete encryption operation.

The present work investigates a narrower question than heterogeneous GPU

[1] Singidunum University, Belgrade, Serbia; E-mail: marko.mihajlovic.23@singimail.rs

acceleration: *how strongly do AES implementation paths and timing boundaries shape CPU encryption measurements on two instruction-capable architectures?* We use already completed experimental campaigns: AES-128 ECB/CTR measurements from Windows x86-64 and Oracle Cloud Arm64, historical AES-CBC experiments covering 128-, 192- and 256-bit keys, and independently validated AES-GCM measurements on Windows. No new experiments were performed for this manuscript. In particular, results from different modes or hosts are not combined into a unified speedup estimate.

The paper has three contributions. First, it documents the magnitude of implementation-path sensitivity on Arm64, including a constrained OpenSSL capability-mask control rather than an unqualified assertion about the AES instruction alone. Second, it distinguishes the native transform from a broader operation envelope on the Windows CPU, using measurements from the same software environment. Third, it analyzes existing AES-CBC and AES-GCM evidence to demonstrate how key length and authentication change throughput within their own explicitly specified workloads. These results motivate a reporting framework in which the cipher mode, implementation, scope, and replication unit accompany each reported performance figure.

This is a retrospective analysis of available datasets, not an attempt to introduce a new cipher or claim a universal processor ranking. All numerical claims are tied to existing experiment outputs; where datasets are not identically instrumented, the limitation is stated explicitly.

2 Background and Related Work

2.1 AES implementation paths and instruction support

AES is specified by FIPS 197 and is instantiated with 128-, 192- or 256-bit keys, requiring 10, 12 or 14 encryption rounds, respectively [1]. Modes of operation govern how successive blocks and counters are processed, not merely which key length is used. CBC encryption has inter-block dependencies, whereas CTR encrypts successive counter blocks and combines the resulting stream with the plaintext [2]. ECB is useful here only as an implementation-path diagnostic, not as a recommendation for sensitive data.

Intel's AES-NI instructions provide an acceleration path for AES rounds [4]. Armv8 implementations can expose hardware AES features through the instruction-set architecture, but availability and actual library dispatch must be established on the measured system. OpenSSL's EVP interface offers a relatively stable cipher abstraction whose internal implementation may differ with providers, CPU features and OpenSSL version [5]. The PyCryptodome package similarly exposes

AES operations through a Python API and native code, with behavior that cannot be assumed identical to EVP [6]. Thus an observed difference is initially an *implementation-path difference*, not proof that a particular instruction is solely responsible.

2.2 Measurement scope and authenticated encryption

Meaningful comparisons require matched payloads, validated ciphertexts and explicit placement of timestamps. A transform-only duration can exclude cipher setup and allocation, while an operation-envelope duration includes a more complete software call. Both may be useful, but they answer different questions and should not be placed into a single speedup column without qualification. Native measurements may also exclude Python entry overhead that another backend includes.

Authenticated encryption introduces a separate distinction. Galois/Counter Mode combines CTR-style encryption with an authentication computation and can verify a tag during decryption [3]. A GCM processing rate therefore measures more functionality than unauthenticated CBC or CTR. The authentication tag, associated data, and encryption context may be outside the byte-count numerator even when their processing is within the timer. Treating these rates as interchangeable AES-throughput figures would be misleading.

The gap addressed here is primarily *reporting and interpretation*: the same CPU, algorithm family and message size can yield dramatically different advertised throughput because of implementation and instrumentation decisions. Rather than proposing a new accelerated implementation, this article organizes existing validated observations around the exact execution boundaries that produced them.

3 Experimental Material and Method

3.1 Platforms and provenance

The retained CPU evidence comes from two distinct environments. The Windows host used an Intel Core i5-10300H (four physical cores, eight logical processors), Python 3.12.10, PyCryptodome 3.23.0 and verified 64-bit OpenSSL 3.6.5 libraries. The Linux Arm64 host was an Oracle Cloud instance on a Neoverse-N1-class CPU, with two available virtual CPUs, Python 3.12.3 and OpenSSL 3.0.13. Architecture, software version, CPU capability and source hashes were preserved in the original experiment records. The two hosts were not matched for frequency, memory hierarchy, virtualization or operating system; hence no intrinsic “Intel versus Arm” hardware speedup is reported.

The campaigns comprised five seeded sessions (2003–2007), known-answer

and differential correctness checks, and separate raw, metadata and summary records. In the CPU library-path campaigns, each size/mode/backend configuration used 20 measured repetitions following five warm-up repetitions. We aggregate session medians rather than pooling all repetitions as if they were independent machines. Historical CBC and GCM figures arose from earlier validated local campaigns; their definitions are retained and are not silently harmonized with the later OpenSSL measurements.

A second Windows workstation (Intel Core i7-11850H, NVIDIA T600) participated in an earlier heterogeneous study. Since the available evidence does not establish an equivalent new OpenSSL CPU-only campaign on that workstation, it is deliberately excluded from the quantitative comparisons of this article. In the same spirit, historical CUDA and page-batching results belong to a distinct submitted manuscript and are not used here.

3.2 Compared operations and execution boundaries

The new CPU-only campaigns implement AES-128 in CTR and ECB modes using PyCryptodome and native OpenSSL EVP. CTR is analyzed as the confidentiality workload; ECB is retained solely as a same-platform implementation diagnostic. The OpenSSL native boundary measures the core EVP Update/Final transform. A separate operation-envelope boundary includes Python-side allocation, setup, bindings, native invocation, conversion to a `bytes` result and relevant cleanup, subject to the explicit wrapper instrumentation. The Python and native transform intervals are not perfectly symmetric: Python invocation/FFI is included in the PyCryptodome path but the outer `ctypes` entry is excluded from the native EVP interval.

On Arm64 an additional `OPENSSL_armcap=0` condition restricts several optimized instruction paths. It is a broad feature-mask control, *not* a selective experiment disabling only AES rounds. The test verifies ciphertext correctness, but the study does not trace executed instructions or isolate NEON, AES and polynomial-multiply effects individually. We therefore describe these measurements as default-versus-masked OpenSSL paths.

The historical CBC measurements compare software AES and an AES-NI-enabled library path for key lengths 128, 192 and 256 bits. Historical GCM measurements use AES-128 and AES-256, 96-bit nonces, a 128-bit tag and a 16-byte associated-data field. Known-answer tests and rejection of modified ciphertext, authentication tag and associated data were required before accepting samples. GCM encryption includes encryption and tag generation, while decryption includes tag verification, and excludes construction of a fresh GCM cipher object from its processing timer.

3.3 Estimators and statistical interpretation

For a payload of B bytes and measured elapsed time t nanoseconds, decimal throughput is

$$P = \frac{10^3 B}{t} \quad [\mathrm{MB/s}]. \tag{1}$$

For implementations A and B measured on the same host and at compatible timing boundaries, the session speedup is the ratio of their median elapsed times:

$$S^{(j)}_{A/B} = \frac{\widetilde{t}^{(j)}_A}{\widetilde{t}^{(j)}_B}, \qquad S_{A/B} = \mathrm{median}^5_{j=1}\, S^{(j)}_{A/B}. \tag{2}$$

Numerical tables in this paper use session-median throughput summaries unless explicitly labeled as a paired time ratio. A median of time ratios need not equal a ratio of separately rounded throughput medians. Five sessions provide descriptive repeatability, not population-level confidence intervals or causality. No outlier exclusion is justified by performance direction. Underlying studies use different scopes; comparisons across them are labeled contextual rather than paired.

4 Results

4.1 Library-path sensitivity on two CPUs

Table 1 condenses the new CPU-only results at 32 MiB in CTR mode. Values are median throughputs of the five session-level medians. On Arm64, the default EVP path achieves approximately 4.03 GB/s at its native transformation boundary; the inspected PyCryptodome and masked-capability paths achieve about 0.21–0.22 GB/s. This more-than-one-order-of-magnitude separation is specific to the measured software paths. It does not show that Arm64 universally outperforms x86-64, and should not be presented as a pure hardware-instruction factor.

The size dependency is shown separately in Fig. 1. On Windows, the small-message value at 1 KiB is dominated by a very short native interval; absolute rates at this scale are extremely sensitive to timer resolution, Python dispatch, and library overhead. Above 1 MiB the native EVP transform approaches a relatively stable multi-GB/s region, whereas the inspected PyCryptodome CTR path remains substantially slower. The Arm64 campaign shows a similarly strong separation of library paths across the tested sizes, but with its own software versions and hardware conditions. The two panels are intentionally not a matched platform speedup comparison.

The Windows transform ratio OpenSSL/PyCryptodome is about 1.9, but the operation-envelope improvement is closer to 1.2. These figures illustrate the

Table 1
AES-128-CTR, 32 MiB: reported throughput by CPU platform, implementation and timing boundary (decimal MB/s).

Host	Implementation	Transform	Operation
Windows i5-10300H	OpenSSL EVP	5537	1479
Windows i5-10300H	PyCryptodome	2877	1202
Oracle Arm64	OpenSSL default	4030	1060
Oracle Arm64	OpenSSL armcap=0	218	—
Oracle Arm64	PyCryptodome	209	—

Transform and operation values are different measured intervals. Oracle operation-envelope throughput is shown for the default EVP path only; unavailable or nonmatching aggregates are not filled in.

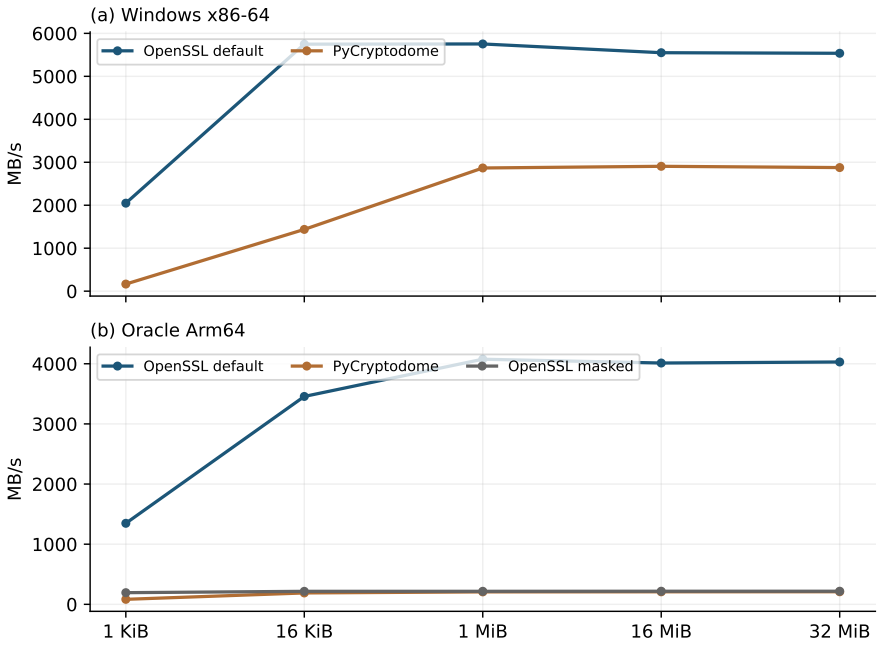


Fig. 1 — Size dependence of the AES-128-CTR transform boundary on (a) Windows x86-64 and (b) Oracle Arm64; original session summaries analyzed separately. The panels illustrate software paths, not an architecture ranking.

sensitivity to timestamp placement. Crucially, some of this sensitivity is a wrapper-level effect: the Python-facing encryption path and native EVP inner interval do not make identical API calls. A system with a different caller, preallocated result buffers or cached contexts may obtain a different relationship.

On Arm64, the default EVP transform rate is approximately 19 times the PyCryptodome path at the same payload size under the observed implementations. The broad feature-mask control closely approaches the PyCryptodome rate. Since the mask removes multiple capabilities, no isolated AES-instruction attribution is possible; differences in context state, provider dispatch, memory copying and language bindings remain alternative or additional explanations.

4.2 Cost of extending the measured boundary

Figure 2 shows the gap between transform-only and broader operation-envelope throughput at 32 MiB on Windows. OpenSSL inner transformation exceeds 5.5 GB/s but the broader calling operation is below 1.5 GB/s. PyCryptodome shows a smaller absolute difference, while its wrapper-level transform remains slower than EVP. These are not contradictory measurements: they include different work. A benchmark that publishes only the inner OpenSSL rate can overstate the throughput a simple Python caller obtains when allocation, cipher setup and copying are repeated for each request.

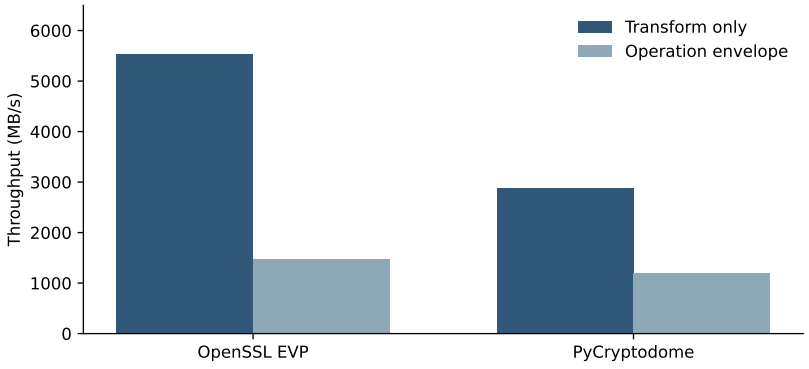


Fig. 2 — Existing Windows measurements of AES-128-CTR at 32 MiB under two different execution boundaries. Both metrics are shown instead of selecting one favorable interval.

The Oracle experiment provides a complementary diagnostic. Its broader default-OpenSSL operation rate at 32 MiB is approximately 1.06 GB/s, compared with 4.03 GB/s at the native inner interval. A separate two-worker run increased aggregate operation-envelope throughput to approximately 2.03 GB/s in the original campaign. The two-worker observation is a system-level scaling diagnostic on that

instance; it cannot be treated as a controlled CPU-core efficiency estimate or as a speedup over a GPU.

4.3 Key length and authenticated workload on Windows

Historical local AES-CBC measurements show that acceleration affects each key length differently. Table 2 reports AES-NI-enabled and software AES throughput for 10 MiB CBC encryption. The ranking is consistent with a higher round count for longer AES keys, but measured ratios also depend on library implementation and setup conventions.

Table 2
Historical 10 MiB CBC encryption on the Windows host (MB/s). These observations are not combined statistically with the later EVP campaign.

Key length	Software	AES-NI path	Reported gain
AES-128	225.1	531.6	2.37×
AES-192	198.5	504.3	2.53×
AES-256	174.9	474.0	2.71×

Although DES and triple DES were also measured in the historical investigation, their role is retrospective only. At 10 MiB, the corresponding software CBC encryption rates were approximately 87.4 and 32.6 MB/s; those values neither identify a modern deployment recommendation nor provide a matched cryptographic-security comparison. We do not expand that dataset into another algorithm-ranking table.

Authentication changes the workload. Table 3 summarizes existing AES-GCM measurements, including tag verification during decryption, at three representative sizes. Small payloads pay a relatively large fixed cost: at 1 KiB, verified decryption is approximately 38 MB/s, whereas at 1 MiB the AES-128 variant exceeds 1.1 GB/s. This is a workload-size effect within one implementation family, not an empirically measured isolated GHASH instruction cost.

Figure 3 uses a logarithmic payload axis to emphasize the transition from per-message overhead to sustained processing. At 16 KiB, authenticated decryption remains substantially slower than encryption, but at 1 MiB the gap is much smaller. This pattern is compatible with amortization of fixed work and mode-specific API behavior; the experiment did not independently time GHASH instructions or cryptographic setup at every size.

The apparent plateau at larger payloads supports an interpretation in which fixed per-message work is amortized, but the measurements do not identify exact allocation, hashing or memory-bandwidth fractions. CBC and GCM values must

Table 3
Existing authenticated AES-GCM throughput on Windows (decimal MB/s; encryption includes tag generation and decryption includes verification).

Payload	Variant	Encrypt	Verify/decrypt
1 KiB	AES-128-GCM	70.6	38.1
1 KiB	AES-256-GCM	70.1	38.0
1 MiB	AES-128-GCM	1148.2	1131.4
1 MiB	AES-256-GCM	1059.3	1059.8
100 MiB	AES-128-GCM	1179.7	1189.3
100 MiB	AES-256-GCM	1100.4	1080.4

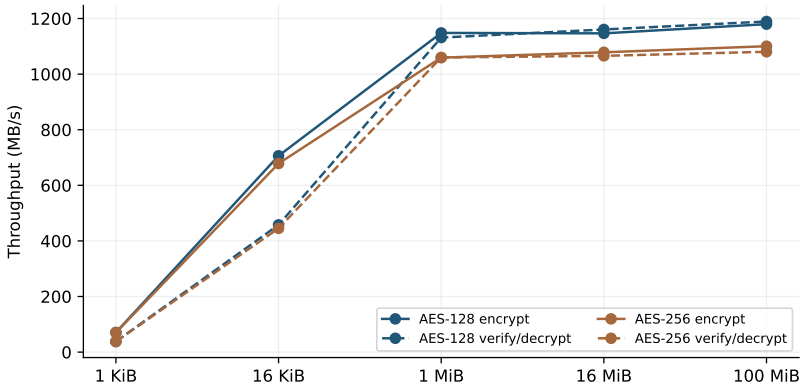


Fig. 3 — Existing Windows AES-GCM encryption and authenticated-decryption throughput for AES-128 and AES-256 over five payload sizes. Lines connect descriptive aggregate values, not a fitted scaling law.

remain separate: GCM returns an authenticated result, while CBC by itself does not.

5 Discussion

5.1 What the results establish

The strongest result is *implementation-path sensitivity under documented timing boundaries*. On Oracle Arm64, default EVP is much faster than the inspected alternative paths for AES-128-CTR at 32 MiB. On Windows, using an inner EVP timer makes OpenSSL appear considerably faster than PyCryptodome; expanding the observation to a broader software operation substantially narrows that advantage. Thus a claim such as "CPU AES achieves 5.5 GB/s" is incomplete without declaring whether the caller is reusing contexts and buffers.

A second observation concerns protected workloads rather than just AES round computations. At 1 KiB, authenticated GCM decryption plus verification can be much slower than at larger sizes when expressed as bytes per second. The AES key length also affects CBC and GCM performance. These data should not be collapsed into one architecture score; they address different cipher modes, key schedules and functional requirements.

5.2 Potential applications and transferability

The most direct engineering application is benchmark and service design for CPU-based encryption components. A storage system, secure message processor or key-management service that repeatedly invokes a cryptographic API should benchmark the actual operation envelope, not only the cipher core. Systems handling large sequential buffers might benefit from native EVP processing and buffer reuse; systems dominated by small authenticated messages may see relatively greater importance of setup and tag verification. This is a plausible design implication, not a measured speedup of a particular storage product or network protocol.

The generalizable element is the methodology, not the numerical ordering of processors. A useful performance report should identify the cipher and mode; key size and nonce/AAD policy; library version and capability dispatch; input residency and buffer policy; transform versus operation timing; and the distribution of session-level observations. The observations from an Arm64 virtual machine and a Windows laptop motivate such reporting across architectures, but do not themselves constitute a matched hardware shootout.

The presence of an additional historical Windows workstation does not remove this limitation. Since that machine lacks a directly comparable archived new EVP

campaign, adding its CPU/GPU values to the present paper would manufacture comparability and overlap with a different submitted study. Likewise, comparing the Oracle CPU directly with a separately timed GPU would be misleading.

5.3 Threats to validity and future research

This retrospective study is constrained by nonidentical APIs, differences in library versions, unpinned operating-system scheduling, DVFS and cloud virtualization. Default OpenSSL capability inspection is not a trace of executed instructions. The Arm capability mask disables multiple instruction classes at once, so a causal claim about any one extension is not warranted. The 32 MiB measurements reuse prepared data, and their cache and memory-allocation behavior may differ from a streaming application. The five seeds produce useful repetition within a platform, not a representative sample of processors. The AES-GCM and CBC campaigns use different timing conventions from the newer CTR experiments.

There are several clearly bounded follow-up studies: (i) use identical native entry points and output-buffer reuse on both architectures; (ii) trace hardware instructions and isolate AES, vector and polynomial-multiply dispatch controls independently; (iii) measure GCM on Arm64 with matched nonce, associated-data and tag-validation policies; (iv) compare single-thread latency and two-worker aggregate throughput under a common process-orchestration protocol; and (v) validate encryption in a real application with request formation, input/output copying and persistent storage or network I/O. Those are future experiments, not results claimed here.

The retained reduced-keyspace and avalanche demonstrations from the larger retrospective have been intentionally excluded. Their principal questions concern educational cryptanalysis and diffusion, rather than CPU encryption costs; including them would weaken the coherence of this implementation-boundary study.

6 Conclusion

Existing validated measurements from Windows x86-64 and Oracle Cloud Arm64 demonstrate that AES throughput is inseparable from the implementation path and the execution interval being timed. A native OpenSSL AES-128-CTR transform reached about 4.03 GB/s on the measured Arm64 system, whereas its inspected alternative paths were near 0.21–0.22 GB/s. On Windows, OpenSSL's 32 MiB inner transform achieved about 5.54 GB/s, but the wider operation envelope reduced reported throughput to about 1.48 GB/s. Additional CBC and GCM evidence illustrates key-length sensitivity and the cost profile of authenticated operations across message sizes. These observations do not rank processor architectures or isolate instruction-level causation. Their practical contribution is a

reproducible way to interpret encryption benchmarks: every performance figure requires a specified cryptographic workload, implementation path, timing scope and replication unit.

References

[1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS 197, Update 1, 2023, doi:10.6028/NIST.FIPS.197-upd1.

[2] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST SP 800-38A, 2001, doi:10.6028/NIST.SP.800-38A.

[3] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: GCM and GMAC*, NIST SP 800-38D, 2007, doi:10.6028/NIST.SP.800-38D.

[4] Intel Corporation, "Intel Advanced Encryption Standard Instructions (AES-NI)," 2012. [Online]. Available: `https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html`.

[5] OpenSSL Project, "EVP symmetric encryption and decryption," OpenSSL documentation. [Online]. Available: `https://docs.openssl.org/3.0/man3/EVP_EncryptInit/`.

[6] PyCryptodome, "AES – Advanced Encryption Standard," documentation. [Online]. Available: `https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html`.

[7] National Bureau of Standards, *DES Modes of Operation*, FIPS PUB 81, 1980.

[8] E. Barker and A. Roginsky, *Transitioning the Use of Cryptographic Algorithms and Key Lengths*, NIST SP 800-131A Rev. 2, 2019, doi:10.6028/NIST.SP.800-131Ar2.